SMART EXAM CHEATING DETECTION SYSTEM

A Database-Driven Approach Using SQL

Detect | Analyze | Prevent

Submitted by:

Naitik Kumar
Group Leader

Vijendra Kumar
Member

Ashish Kumar Singh
Member

Sunny Kumar
Member

Course: DBMS Project | **Academic Year:** 2025-26 | **Department:** Computer Science | **University:** IILM University

PROJECT OVERVIEW

S.No	Section	Coverage
1.	Introduction	What is the project? What problem does it solve?
2.	ER-Diagram	It represents a visual representation of entities, their attributes, and the relationships between them in a database system.
3.	Problem Statement	Why existing systems fail at detecting cheating
4.	Literature Review	Review of existing research and techniques used for cheating detection systems
5.	Methodology	Database design, all tasks completed step by step
6.	Conclusion	Summary and real-world applicability

1. INTRODUCTION

- Detecting cheating in online examinations is a major challenge, especially with a large number of students.
- Most existing systems focus only on answer evaluation and marks calculation, ignoring student behavior.
- Due to the lack of behavior monitoring, cheating often goes undetected.
- To address this issue, a **Smart Exam Cheating Detection System** was developed using SQL and DBMS concepts.
- The system uses MariaDB/MySQL to automatically detect cheating behaviors without using any AI or Machine Learning.

Technology Used

- RDBMS: MariaDB (via XAMPP)
- Language: SQL — DDL, DML, DQL
- Features: Views, Triggers, Joins, Subqueries, Aggregate Functions

System Overview

- 6 normalized tables
- 10 students, 4 exams sample data
- 26 SQL queries covering all concepts
- 7 cheating flags

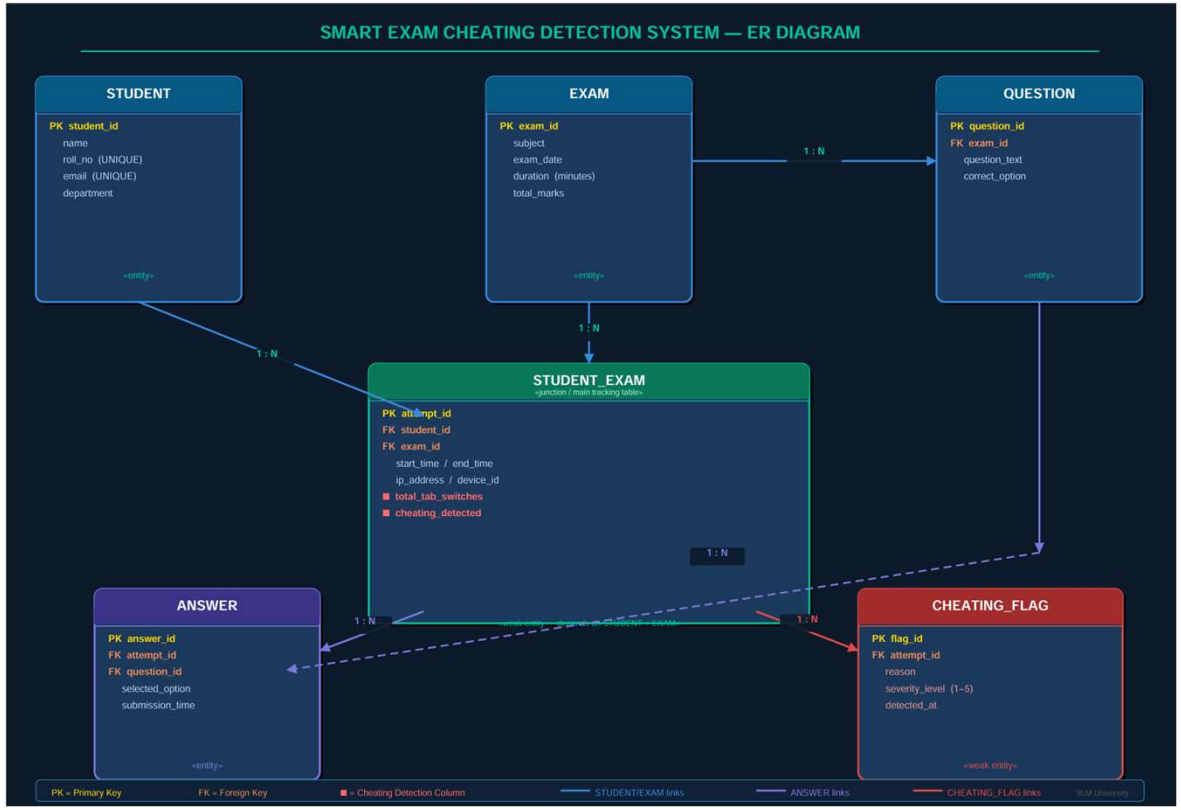
1. INTRODUCTION

Database Tables

S. No.	Table	Purpose
1.	STUDENT	Student information & department
2.	EXAM	Exam schedule & marks
3.	QUESTION	MCQ question & marks
4.	STUDENT_EXAM	Attempt tracking (IP, Tab switches)
5.	ANSWER	Student responses per question
6.	CHEATING_FLAG	Detect violations + severity

The database was initially designed with raw tables and then systematically normalized to 1NF, 2NF, 3NF. This process removed data redundancy, eliminated dependencies, and ensured better data integrity and efficiency.

2. ER-DIAGRAM



The ER diagram illustrates how different tables are connected to manage and analyze exam data effectively.

3. PROBLEM STATEMENT

- Most online exam systems only focus on storing answers and calculating marks. They do not track or analyze how a student behaves during the exam, which makes cheating difficult to detect. Also, manual monitoring is not practical in large exams.
- There is also no proper system to automatically detect common cheating methods like copying answers, using the same internet connection, or frequently switching tabs during the exam.
- To solve this, our system uses SQL queries to analyze student activity data and detect cheating patterns automatically. It identifies things like shared IP addresses, excessive tab switching, and identical answer patterns.
- Each suspicious activity is properly recorded with a reason.
- The main idea is that cheating always leaves digital traces, and by analyzing this data using SQL, we can detect such behavior quickly and efficiently.

3. PROBLEM STATEMENT

The Gap in Online Exam Systems vs Our Solution

Problem in Existing Systems	Our Solution
Most systems only store answers and calculate marks. They do not analyze student behavior.	Our system analyzes student activity data during exams using SQL.
Manual monitoring is not practical for large exams (one invigilator cannot monitor hundreds of students).	SQL queries automatically check all student data within seconds.
No proper way to detect cheating methods like copying answers, shared IPs, or tab switching.	Detects cheating using patterns like shared IP, high tab switching, and identical answers.
Decisions are often based on suspicion without proper proof.	Each cheating case is recorded with reason, severity level, and linked to a specific attempt.

4. LITERATURE REVIEW

Key Concepts Applied:

- In this project, we used some basic and important DBMS concepts as our foundation.
- First, we followed the **Relational Database Model**, where data is organized in the form of tables and connected using keys. This helps in maintaining proper structure and relationships between data.
- We use **SQL not just for storing data, but for analysis**. Functions like COUNT, AVG, GROUP BY, and subqueries help in identifying patterns in student behavior.
- Next, we applied **Normalization (1NF, 2NF & 3NF)** step by step to remove data redundancy and make the database clean and efficient. After normalization, there are no unnecessary dependencies in the database.
- Finally, if our system observe any suspicious or unusual activities like high tab switching or shared IP addresses are treated as suspicious behavior.

4. LITERATURE REVIEW

DBMS Concepts Used:

Concept	How we use it
Primary Key	Unique ID for each record
Foreign Key	Establishes relationships between tables
Constraints	Ensures data integrity (NOT NULL, UNIQUE, etc.)
JOIN	Combines data from multiple tables
Subquery	Helps in conditional data filtering
Aggregate Functions	Used for analysis (COUNT, AVG, MAX)
HAVING	Filters grouped data
VIEW	Stores reusable query results
3NF	Removes redundancy and ensures efficiency

5. METHODOLOGY

The system was developed in **three** main phases:

Phase	Activity	Output
Phase 1	ER Design + Problem Understanding	Structured database schema
Phase 2	SQL Implementation (CRUD + Queries)	Working database + results
Phase 3	Normalization (1NF–3NF)	Optimized database

5. METHODOLOGY

The system was developed in **three** main phases:

◆ Phase 1: Database Design

• Identified entities: **Student, Exam, Question, Attempt, Answer, Flag**
• Designed **ER Diagram** to define relationships
• Applied **Primary Key & Foreign Key** constraints
• Structured schema to support **cheating detection system**

◆ Phase 2: SQL Implementation

• Created tables using **CREATE TABLE (MariaDB)**
• Performed operations:
  • **INSERT, UPDATE, DELETE**
  • **SELECT, JOIN, GROUP BY**
• Executed **26 SQL queries** for data analysis
• Detected patterns like:
  • **Shared IP addresses**
  • **Identical answers**
  • **High tab switching**

5. METHODOLOGY

Phase 3 – Normalization

- **Normalization Process (UNF → 1NF → 2NF → 3NF)**
  - **UNF** → Data stored in single table, high redundancy
  - **1NF** → Atomic values, no repeating groups
  - **2NF** → Removed partial dependency
  - **3NF** → Removed transitive dependency

- **Functional Dependencies**
  - **student_id** → name, roll_no, email
  - **exam_id** → subject, date, marks
  - **attempt_id** → student_id, exam_id, ip_address
  - **(attempt_id, question_id)** → selected_option
  - **flag_id** → attempt_id, reason

- **Final Outcome**
  - Database normalized up to **3NF**
  - **Redundancy removed**
  - Improved data consistency & efficiency

5. METHODOLOGY

Task Breakdown — Normalization (1NF→2NF→3NF)

- **UNF (Unnormalized Form):**
  At the beginning, all the data was stored in a single table with around 27 columns. Which caused a lot of repetition, like student names being repeated for every answer, making the data messy and hard to manage.

- **1NF (First Normal Form):**
  In this step, we made sure that each column contains only one value (atomic values).
  We also removed repeating groups and used a composite primary key to uniquely identify each record. This made the data more structured.

- **2NF (Second Normal Form):**
  In 2NF, we divided the data into multiple tables. Each attribute was made dependent only on its primary key, not on part of it.
  For example, student details were moved to a separate STUDENT table.This reduced redundancy.

- **3NF (Third Normal Form):**
  Finally, we removed transitive dependencies. This means no non-key attribute depends on another non-key attribute. After this step, the database became fully clean, organized, and efficient.

5. METHODOLOGY

Task Breakdown — Normalization (1NF→2NF→3NF)

Normal Form	Key Rule	Result
UNF	No specific rule	1 table with many columns and high redundancy
1NF	Atomic values, no repeating groups	Data organized with single values in each cell
2NF	No partial dependencies	Data divided into multiple tables
3NF	No transitive dependencies	Clean and well-structured database

Functional Dependencies:

- **student_id** → name, roll_no, email, department
- **exam_id** → subject, exam_date, duration, total_marks
- **attempt_id** → student_id, exam_id, ip_address, tab_switches
- **(attempt_id, question_id)** → selected_option, submission_time
- **flag_id** → attempt_id, reason, severity_level

KEY RESULT

In this project, we used SQL queries to detect different types of cheating during online exams.
We found **three** main cheating patterns:

1. Shared IP Address

- Some students logged in using the same IP address
- This indicates possible collaboration
- **Examples:**
  - Arjun & Priya (same IP)
  - Vikram & Anjali (same IP)

2. Excessive Tab Switching

- Students who switched tabs too frequently were marked suspicious
- Threshold was set (more than 10 switches)
- **Examples:**
  - Kavya Nair — 22 switches (High severity)
  - Sneha Patel — 18 switches
  - Priya Verma — 15 switches

3. Identical Answers

- Some students submitted exactly the same answers at similar times
- Indicates direct copying
- **Example:**
  - Priya Verma copied answers from Arjun Sharma (High severity)

Summary:

Total Students: 10
Flagged Students: 5 (50%)
Total Flags: 7
Patterns Detected: 3
Clean Students: 5

6. CONCLUSION

What We Achieved:

- We designed a clean and well-structured database with 6 tables (up to 3NF)
- Successfully created and tested the database in MariaDB
- Detected cheating patterns like shared IP, high tab switching, and identical answers
- Wrote 26 SQL queries covering important DBMS concepts
- Applied normalization step by step from UNF to 3NF

Future Scope:

- Add triggers for real-time cheating detection
- Develop a web dashboard for easy monitoring
- Integrate with proctoring systems for live alerts